asio处理粘包的简易方式

Posted on 2023-05-27 10:20:12 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2QMCMAfo5N6IUaHceKcSmlwUaZs)

简易方式

之前我们介绍了通过async_read_some函数监听读事件，并且绑定了读事件的回调函数HandleRead

```
1. _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH), std::bind(&CSession::HandleRead, this,
2. std::placeholders::_1, std::placeholders::_2, SharedSelf()));
```

async_read_some 这个函数的特点是只要对端发数据，服务器接收到数据，即使没有收全对端发送的数据也会触发HandleRead函数，所以我们会在HandleRead回调函数里判断接收的字节数，接收的数据可能不满足头部长度，可能大于头部长度但小于消息体的长度，可能大于消息体的长度，还可能大于多个消息体的长度，所以要切包等，这些逻辑写起来很复杂，所以我们可以通过读取指定字节数，直到读完这些字节才触发回调函数，那么可以采用async_read函数，这个函数指定读取指定字节数，只有完全读完才会触发回调函数。

获取头部数据

我们可以读取指定的头部长度，大小为HEAD_LENGTH字节数，只有读完HEAD_LENGTH字节才触发HandleReadHead函数

```
1. void CSession::Start(){
2.     _recv_head_node->Clear();
3.     boost::asio::async_read(_socket, boost::asio::buffer(_recv_head_node->_data, HEAD_LENGTH), std::bind(&CSession::HandleReadHead, this,
4.         std::placeholders::_1, std::placeholders::_2, SharedSelf()));
5. }
```

这样我们可以直接在HandleReadHead函数内处理头部信息

```
1. void CSession::HandleReadHead(const boost::system::error_code& error, size_t  bytes_trans
   ferred, std::shared_ptr<CSession> shared_self) {
2.     if (!error) {
3.         if (bytes_transferred < HEAD_LENGTH) {
4.             cout << "read head lenth error";
5.             Close();
6.             _server->ClearSession(_uuid);
7.             return;
8.         }
9.
10.        //头部接收完，解析头部
11.        short data_len = 0;
12.        memcpy(&data_len, _recv_head_node->_data, HEAD_LENGTH);
13.        cout << "data_len is " << data_len << endl;
14.        //此处省略字节序转换
15.        // ...
16.        //头部长度非法
17.        if (data_len > MAX_LENGTH) {
18.            std::cout << "invalid data length is " << data_len << endl;
19.            _server->ClearSession(_uuid);
20.            return;
21.        }
22.
23.        _recv_msg_node= make_shared<MsgNode>(data_len);
24.        boost::asio::async_read(_socket, boost::asio::buffer(_recv_msg_node->_data, _recv_msg
   _node->_total_len),
25.            std::bind(&CSession::HandleReadMsg, this,
26.            std::placeholders::_1, std::placeholders::_2, SharedSelf()));
27.    }
28.    else {
29.        std::cout << "handle read failed, error is " << error.what() << endl;
30.        Close();
31.        _server->ClearSession(_uuid);
32.    }
33. }
```

接下来根据头部内存储的消息体长度，获取指定长度的消息体数据，所以再次调用async_read，指定读取 `_recv_msg_node->_total_len` 长度，然后触发HandleReadMsg函数

获取消息体

HandleReadMsg函数内解析消息体，解析完成后打印收到的消息，接下来继续监听读事件，监听读取指定头部大小字节，触发HandleReadHead函数， 然后再在HandleReadHead内继续监听读事件，获取消息体长度数据后触发HandleReadMsg函数，从而达到循环监听的目的。

```
1. void CSession::HandleReadMsg(const boost::system::error_code& error, size_t  bytes_transfe
   rred,
2.     std::shared_ptr<CSession> shared_self) {
3.     if (!error) {
4.         PrintRecvData(_data, bytes_transferred);
5.         std::chrono::milliseconds dura(2000);
6.         std::this_thread::sleep_for(dura);
7.         _recv_msg_node->_data[_recv_msg_node->_total_len] = '\0';
8.         cout << "receive data is " << _recv_msg_node->_data << endl;
9.         Send(_recv_msg_node->_data, _recv_msg_node->_total_len);
10.        //再次接收头部数据
11.        _recv_head_node->Clear();
12.        boost::asio::async_read(_socket, boost::asio::buffer(_recv_head_node->_data, HEAD_LE
   NGTH),
13.            std::bind(&CSession::HandleReadHead, this, std::placeholders::_1, std::placeholder
   s::_2,
14.                SharedSelf()));
15.    }
16.    else {
17.        cout << "handle read msg failed,  error is " << error.what() << endl;
18.        Close();
19.        _server->ClearSession(_uuid);
20.    }
21. }
```

总结

本文介绍了如何使用async_read函数，监听读事件获取指定字节数才触发回调函数，用这种办法处理粘包问题很简单。
源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2692) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(510) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)